ARTIFICIAL INTELLIGENCE & DATA MINING

For

Mini Project

Group 69.

SUN CHENYU

CHENG YI

ZHU YANGKE

Date 2026/10/13

Table of Contents

1. LITERATURE SURVEY 1

2. DATASET AND PRE-PROCESSING (A) 2

2.1 Dataset composition and split 2
2.2 Deterministic pre-processing 2
2.3 Training-only augmentation 2
2.4 B extension: shared data protocol and resolution control 3

3. BASELINE CLASSIFIER (B) 4

3.1 CNN architecture 4
3.2 Loss function and training strategy 4
3.3 Reproducibility and running instructions 5
3.4 B extension: pretrained ResNet18 classifier 5
3.5 B extension: optimization and implementation 6
3.6 B extension: environment and running instructions 7

4. PARAMETERS AND SETTINGS (C) 9

4.1 Controlled parameter experiments 9
4.2 Discussion and final setting 9
4.3 B extension: parameter choices and matched 20-epoch controls 10

5. VALIDATION ACCURACY AND TEST PREDICTIONS (D) 12

5.1 B extension: validation results and checkpoint selection 12
5.2 B extension: test prediction and submission format 13

6. CORRECT AND INCORRECT SAMPLE ANALYSIS (E) 14

6.1 Overall validation behaviour 14
6.2 Selected correct and incorrect cases 14
6.3 B extension: labelled validation cases 15
6.4 B extension: qualitative inspection of unlabelled test images 16

7. MODEL AND DATA PROCESSING COMPARISON (F) 18

7.1 B extension: model and augmentation effects at 128 × 128 18
7.2 B extension: input resolution and relation to the A baseline 19
7.3 B extension: reproducibility and limitations 19
7.4 B contribution and handoff 20
7.5 References for the B extension 20

8. CIFAR-10 MULTI-CATEGORY CLASSIFICATION (G) 21

9. CIFAR-10 CLASS IMBALANCE (H) 22

1. Literature Survey

2. Dataset and Pre-processing (a)

2.1 Dataset composition and split

The dataset contains 25,500 RGB images. Of these, 20,000 are used for training, 5,000 for validation and 500 form the unlabelled test set. The labelled splits are balanced between cats and dogs; cats use label 0 and dogs label 1. All three parameter experiments use the same seed-42 manifest, whose SHA-256 hash is 4cb1a6c0a99bc1133941f2b9553d272775a53eb00478fd4b47fc5061a4c21104. Keeping this file list fixed ensures that every run is trained and evaluated on the same images.

Split	Cat	Dog	Unlabelled	Total
Training	10,000	10,000	—	20,000
Validation	2,500	2,500	—	5,000
Test	—	—	500	500

Table 2.1 Dataset composition used by the baseline pipeline

2.2 Deterministic pre-processing

Each image is first converted to RGB and resized to 128 × 128 pixels with bilinear interpolation. After conversion to a tensor, the three channels are normalized using mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). The validation pipeline stops at these deterministic steps. Random augmentation is reserved for training, so the validation score is measured on a stable set of inputs.

2.3 Training-only augmentation

For training, the pipeline may flip an image horizontally ($p = 0.5$) and rotate it by a random angle between $-15°$ and $+15°$. Rotation uses bilinear interpolation without enlarging the canvas; any exposed border is filled with RGB (124, 116, 104), which is close to the dataset's normalized mean. The operations run in this order: resize, flip, rotate, convert to tensor and normalize. These changes alter pose and orientation without changing whether the image contains a cat or a dog.

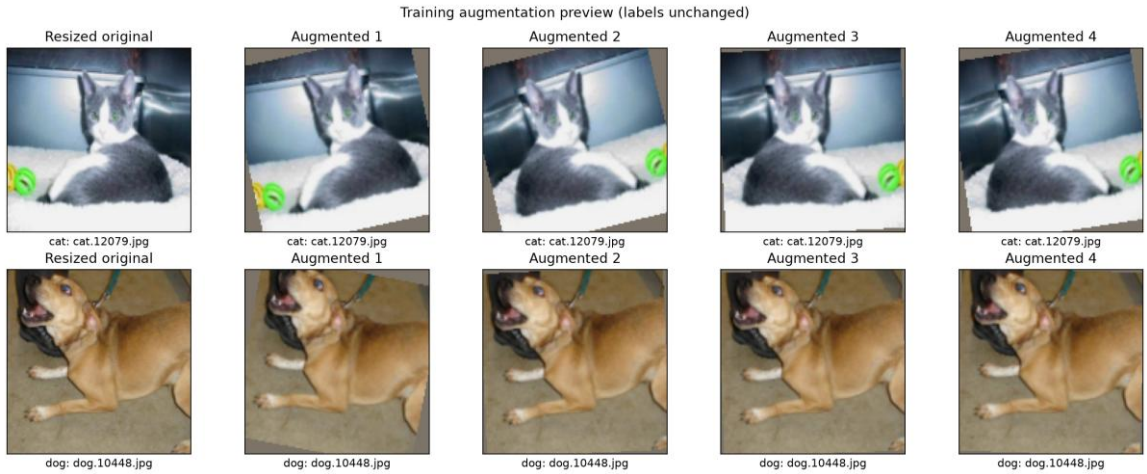


Figure 2.1 Preview of the training-only augmentation pipeline

2.4 B extension: shared data protocol and resolution control

The transfer-learning experiments use the same fixed course split and label map as the baseline: 20,000 training images, 5,000 validation images and 500 unlabelled test images; cat = 0 and dog = 1. Only manifest entries assigned to the training directory are used for gradient updates; the known exact cross-split duplicate is disclosed below. All five new runs use the complete training and validation sets, the shared manifest and seed 42. At 128 × 128, the CNN and ResNet18 use identical input preparation and the same training-only augmentation setting. A separate 224 × 224 ResNet18 run changes only input resolution relative to its augmented 128 × 128 counterpart.

Training augmentation consists of a horizontal flip with probability 0.5 and a uniformly sampled rotation from −15° to +15°. Validation and test preprocessing are deterministic: RGB conversion, bilinear square resize, tensor conversion and ImageNet channel normalization. Square resizing is a deliberate project protocol; it differs from the official ResNet18 weight transform, which resizes the shorter side to 256 and centre-crops to 224 [B2].

An exact-byte duplicate audit found five duplicate groups, including one cross-split pair: train/cat/cat.2339.jpg and val/cat/cat.12408.jpg. The official split is retained for the primary comparison. Supplementary validation accuracy excludes only this one validation image (n = 4,999); it does not remove the training copy or constitute a fully deduplicated retraining experiment. Perceptual duplicates were not audited. The full dataset archive and manifest hashes are supplied with the experiment evidence.

3. Baseline Classifier (b)

3.1 CNN architecture

The baseline is a small CNN trained from scratch. It has three feature-extraction blocks, each made up of a 3 × 3 padded convolution, batch normalization, ReLU and 2 × 2 max pooling. The channel count rises from 3 to 32, 64 and 128, while pooling halves the spatial resolution at each stage. Adaptive average pooling reduces the last feature map to 128 values. Dropout is then applied before a linear layer outputs two logits. In total, the network has 93,730 trainable parameters.

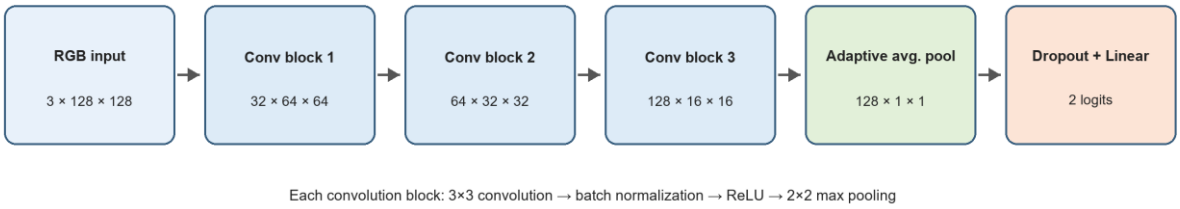


Figure 3.1 Baseline CNN architecture and tensor dimensions

Stage	Operation	Output shape
Input	RGB image	3 × 128 × 128
Block 1	Conv 3×3, BN, ReLU, MaxPool 2×2	32 × 64 × 64
Block 2	Conv 3×3, BN, ReLU, MaxPool 2×2	64 × 32 × 32
Block 3	Conv 3×3, BN, ReLU, MaxPool 2×2	128 × 16 × 16
Pooling	Adaptive average pooling	128 × 1 × 1
Classifier	Flatten, dropout 0.3, linear 128→2	2 logits

Table 3.1 Layer sequence and output dimensions

3.2 Loss function and training strategy

Training minimizes two-class cross-entropy computed from the two output logits. AdamW is used for 20 epochs with a learning rate of 0.001 and weight decay of 0.0001. The batch size is 32 and dropout is 0.3. A scheduler was deliberately omitted to keep the baseline simple and easier to reproduce. Validation runs after every epoch. The checkpoint is saved to best_model.pt whenever validation accuracy improves, while last_model.pt records the final epoch.

Setting	Value	Reason
Input size	128 × 128	Limits computation while retaining coarse shape and texture
Optimizer	AdamW	Adaptive updates with explicit weight decay
Learning rate	0.001	Best validation result among the tested rates
Batch size	32	More parameter updates per epoch than batch 64
Weight decay	0.0001	Mild regularization for a small CNN

Setting	Value	Reason
Dropout	0.3	Reduces reliance on individual pooled features
Epochs	20	Long enough for convergence in all controlled runs
Random seed	42	Reproducible split, augmentation sequence and initialization

Table 3.2 Final baseline training settings

3.3 Reproducibility and running instructions

The completed run used Python 3.11.9, PyTorch 2.14.1+cu130 and torchvision 0.29.1+cu130 on an NVIDIA GeForce RTX 5060 Laptop GPU. The command below reproduces the baseline from the project root. Because the training script does not overwrite an existing run, a fresh output directory is required when repeating the experiment.

```
.\.venv\Scripts\python.exe -m src.train --output-dir runs/baseline_reproduction --epochs 20
--batch-size 32 --lr 0.001 --seed 42 --augment --augmentation-config
configs/augmentation.json --show-plots
```

Evaluation reloads best_model.pt and reuses the validation manifest stored with the run. It writes the loss and accuracy, per-image class probabilities, a confusion matrix, and examples of correct and incorrect predictions.

```
.\.venv\Scripts\python.exe -m src.evaluate --checkpoint
runs/baseline_reproduction/best_model.pt --output-dir runs/baseline_reproduction/evaluation
--show-plots
```

3.4 B extension: pretrained ResNet18 classifier

ResNet18 [B1] is initialized with torchvision ResNet18_Weights.IMAGENET1K_V1 [B2]. Its 1,000-class classifier is replaced by a newly initialized linear layer with 512 inputs and two output logits. The first convolution, pooling and residual stages layer1–layer3 are frozen; layer4 and the new classifier are fine-tuned. Crucially, the frozen stages remain in evaluation mode during training, so their BatchNorm running statistics are also fixed. The model contains 11,177,538 parameters, of which 8,394,754 are trainable. This comparison measures the combined change in architecture, pretrained representation and fine-tuning strategy; it does not isolate pretraining from model capacity.

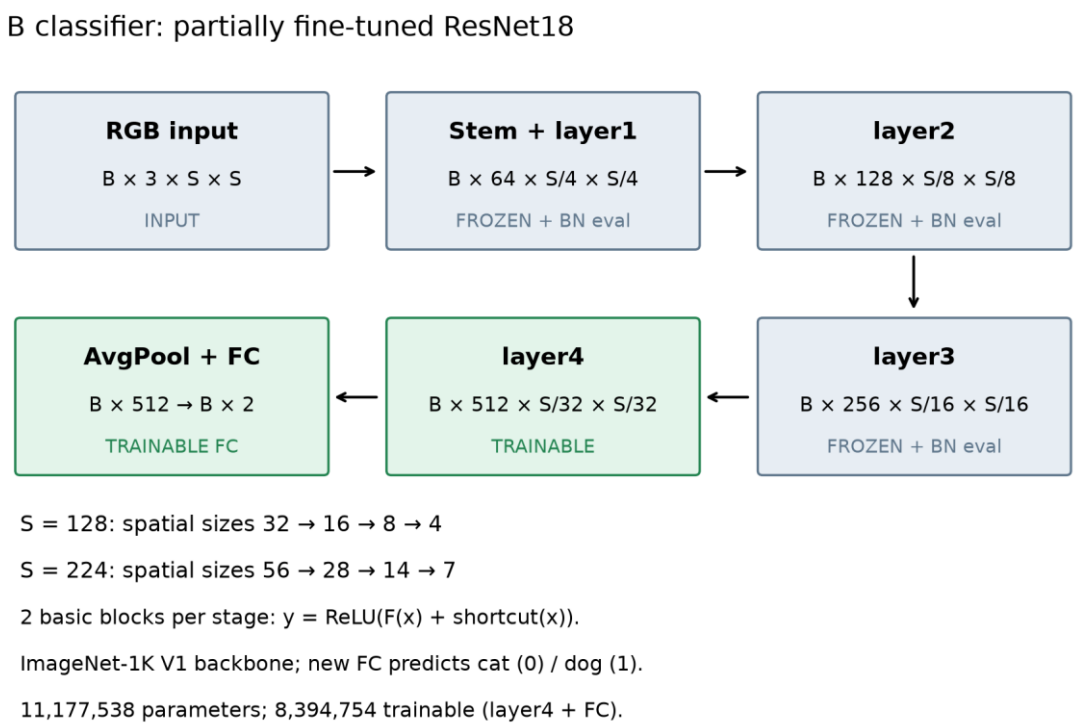


Figure 3.2 B classifier architecture, tensor dimensions and freeze policy. B denotes batch size; S is 128 or 224.

For an input B × 3 × S × S, the residual stages output 64 × S/4 × S/4, 128 × S/8 × S/8, 256 × S/16 × S/16 and 512 × S/32 × S/32 feature maps. Global average pooling produces B × 512 features, and the new linear head produces B × 2 logits. Cross-entropy is applied directly to logits; softmax is used only when reporting probabilities. The prediction is the index of the larger logit.

3.5 B extension: optimization and implementation

All new experiments run for exactly 20 epochs from a fresh model and optimizer state (with the official pretrained backbone for ResNet18). For ResNet18, AdamW uses a classifier learning rate of 1×10^{-3} and a layer4 learning rate of 1×10^{-4}, with weight decay 1×10^{-4}. The lower backbone rate limits disruption of pretrained features while allowing the classifier to adapt faster. The CNN control uses AdamW at 1×10^{-3} and its original dropout of 0.3. Batch size is 32; no scheduler, early stopping, mixed precision or test-time augmentation is used. Every run saves the final checkpoint and the checkpoint with maximum validation accuracy, breaking ties by lower validation loss. Thus a 20-epoch training budget can select a checkpoint earlier than epoch 20.

The following executable class excerpt is taken from b_module/model.py. Its train() override is required because the shared training engine calls model.train() each epoch. Complete source files, not only this excerpt, are included in the companion B reproduction package.

```
"""ResNet18 with explicit freeze policy, including BatchNorm running statistics."""
from torch import nn
from torchvision.models import resnet18, ResNet18_Weights


class TransferResNet18(nn.Module):
    def __init__(self, mode='layer4', pretrained=False):
        super().__init__()
        if mode not in ('head', 'layer4', 'all'):
            raise ValueError(mode)
        self.mode = mode
        self.network = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1 if pretrained else None)
        self.network.fc = nn.Linear(self.network.fc.in_features, 2)
        for name, parameter in self.network.named_parameters():
            parameter.requires_grad = (mode == 'all' or name.startswith('fc.') or
                                       (mode == 'layer4' and name.startswith('layer4.')))

    def train(self, mode=True):
        super().train(mode)
        if mode and self.mode != 'all':
            # src.engine calls model.train() each epoch. Keep frozen BN statistics fixed.
            self.network.eval()
            self.network.fc.train()
            if self.mode == 'layer4':
                self.network.layer4.train()
        return self

    def forward(self, inputs):
        return self.network(inputs)
```

3.6 B extension: environment and running instructions

The new controlled runs used Python 3.12.3, PyTorch 2.14.1+cu130, torchvision 0.29.1+cu130 and an NVIDIA GeForce RTX 2060 (6 GB). All five runs used the same environment, four data-loader workers and four CPU threads. Python, NumPy, PyTorch and data-loader generators are seeded. PyTorch deterministic algorithms are enabled with warn_only=True, so fixed seeds improve reproducibility but do not guarantee bitwise equality across hardware or software versions. The exact package lock, configuration, source hashes and per-epoch histories are included in the evidence package.

As an observed repeatability check on the recorded RTX 2060 environment, the first 10 epochs of all four fresh 128 × 128 runs exactly reproduced the training/validation loss and accuracy trajectories of their earlier 10-epoch counterparts. This check does not imply bitwise reproducibility on another machine, and it does not turn the new one-seed matrix into independent multi-seed repetitions.

For the recorded Linux/WSL environment, from the supplied reproduction package root (the directory containing src/, b_module/ and reproduce_20epoch.py), create and activate a Python environment, install the locked packages and point DATA_ROOT at the extracted course datasets folder. DATA_ROOT must contain train/, val/ and test/. A CUDA-capable GPU is recommended; CPU execution is available but will be much slower. The official pretrained weight file is downloaded on first use and is never silently replaced by random weights.

```
python3.12 -m venv .venv_b20
source .venv_b20/bin/activate
python -m pip install -r environment-lock.txt
python -m b_module.preflight --data-dir "DATA_ROOT" --sample-submission
report_assets/sampleSubmission.csv --verify-images
python reproduce_20epoch.py --data-dir "DATA_ROOT" --output-dir "runs/b20_reproduction" --device
cuda --num-workers 4
```

The full environment lock describes Linux/WSL; native Windows users need the corresponding platform-specific PyTorch installation rather than applying Linux-only dependency wheels. Replace DATA_ROOT with the actual path; keep quotation marks if it contains spaces. The output directory must be new. The reproduction script trains all five controls, independently reloads each best checkpoint, compares results, selects the best ResNet18 candidate using validation data alone and generates the 500-row test submission. To inspect an individual run, use the module commands below; replace SELECTED_RUN with the selected run directory. Evaluation and prediction output directories must also be new.

```
python -m b_module.evaluate --checkpoint "SELECTED_RUN/best_model.pt" --data-dir "DATA_ROOT" --
output-dir "runs/b20_check" --device cuda --sensitivity-exclude val/cat/cat.12408.jpg
python -m b_module.predict --checkpoint "SELECTED_RUN/best_model.pt" --test-dir "DATA_ROOT/test"
--sample-submission report_assets/sampleSubmission.csv --output-dir "runs/b20_predictions" --
device cuda
```

The unchanged shared APIs are src.data.build_loaders, src.engine.train_one_epoch and src.engine.evaluate. Model-specific behavior is isolated in b_module. The package README maps the code, checkpoints, histories, figures, predictions and provenance to the report. The raw runs retain their original local paths in config.json for audit; reproduction uses the DATA_ROOT and new output directory supplied by the user.

4. Parameters and Settings (c)

4.1 Controlled parameter experiments

Three 20-epoch runs were used for the parameter comparison. The manifest, random seed, network, image size, augmentation, optimizer, weight decay and dropout stay fixed across all runs. The reference uses a learning rate of 0.001 and batch size 32. The second run changes only the learning rate to 0.0003, and the third changes only the batch size to 64. Varying one setting at a time makes the comparison easier to interpret.

Run	Learning rate	Batch size	Best epoch	Best val. accuracy	Val. loss at best epoch
Reference	0.001	32	18	83.34%	0.3857
Lower learning rate	0.0003	32	19	80.00%	0.4525
Larger batch	0.001	64	19	81.84%	0.4198

Table 4.1 Parameter comparison using the same split and random seed

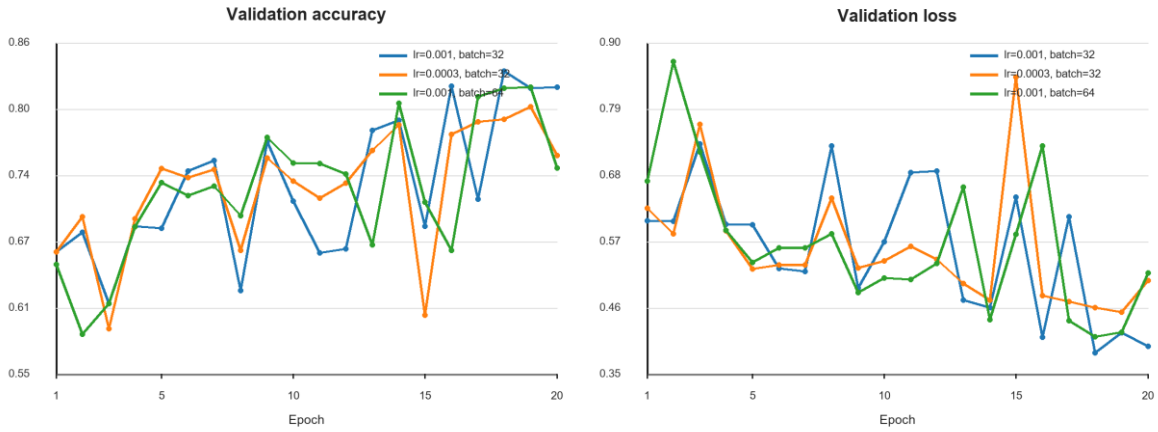


Figure 4.1 Validation trajectories for the parameter experiments

4.2 Discussion and final setting

With the learning rate reduced to 0.0003, training progressed more gradually but reached a best validation accuracy of 80.00%, 3.34 percentage points below the reference. Twenty epochs may not have been enough for the smaller updates to reach a comparable solution. A batch size of 64 finished faster, but its best validation accuracy was 81.84%, 1.50 points below the reference. The larger batches therefore saved some training time at the cost of validation performance.

The final baseline uses a learning rate of 0.001 and batch size 32. Among the tested settings, it produced the highest validation accuracy (83.34%) and the lowest loss at the selected checkpoint (0.3857). Validation accuracy still moved noticeably between epochs even as training loss fell, which is why the best checkpoint is preferable to the final one. Since each configuration was run with a single seed, these

results justify the project setting but do not establish statistical superiority.

4.3 B extension: parameter choices and matched 20-epoch controls

An earlier 10-epoch exploration compared head-only training, partial layer4 fine-tuning, lower learning rates and input resolution. At 128 × 128 with augmentation, head-only training reached 96.34%, partial layer4 fine-tuning reached 97.38%, and reducing both learning rates to 3×10^{-4} / 3×10^{-5} reached 97.22%. The augmented 224 × 224 setting reached 98.96%. These exploratory observations motivated partial fine-tuning and the chosen rates; they are not budget-matched evidence against the 20-epoch A baseline. They also reflect selection on the same validation set.

To remove the 10-versus-20 epoch mismatch in the main comparison, five fresh 20-epoch runs were completed. The first four form a 2 × 2 comparison of model family and augmentation at 128 × 128. The fifth evaluates resolution within the augmented ResNet18 configuration. Each run processes 20,000 training images per epoch, with 625 optimizer steps per epoch (12,500 in total). This matches the number of training passes and updates, not GPU time or model capacity. The original A study in Table 4.1 is retained as reported; the new CNN controls are separate runs on the B experiment environment.

Run	Classifier / input	Aug.	Head / backbone LR	Epochs
C0	CNN / 128 × 128	No	0.001 / n.a.	20
C1	CNN / 128 × 128	Yes	0.001 / n.a.	20
R0	ResNet18 / 128 × 128	No	0.001 / 0.0001	20
R1	ResNet18 / 128 × 128	Yes	0.001 / 0.0001	20
R2	ResNet18 / 224 × 224	Yes	0.001 / 0.0001	20

Table 4.2 New 20-epoch matrix. All runs use seed 42, batch size 32, AdamW, weight decay 0.0001 and the same manifest.

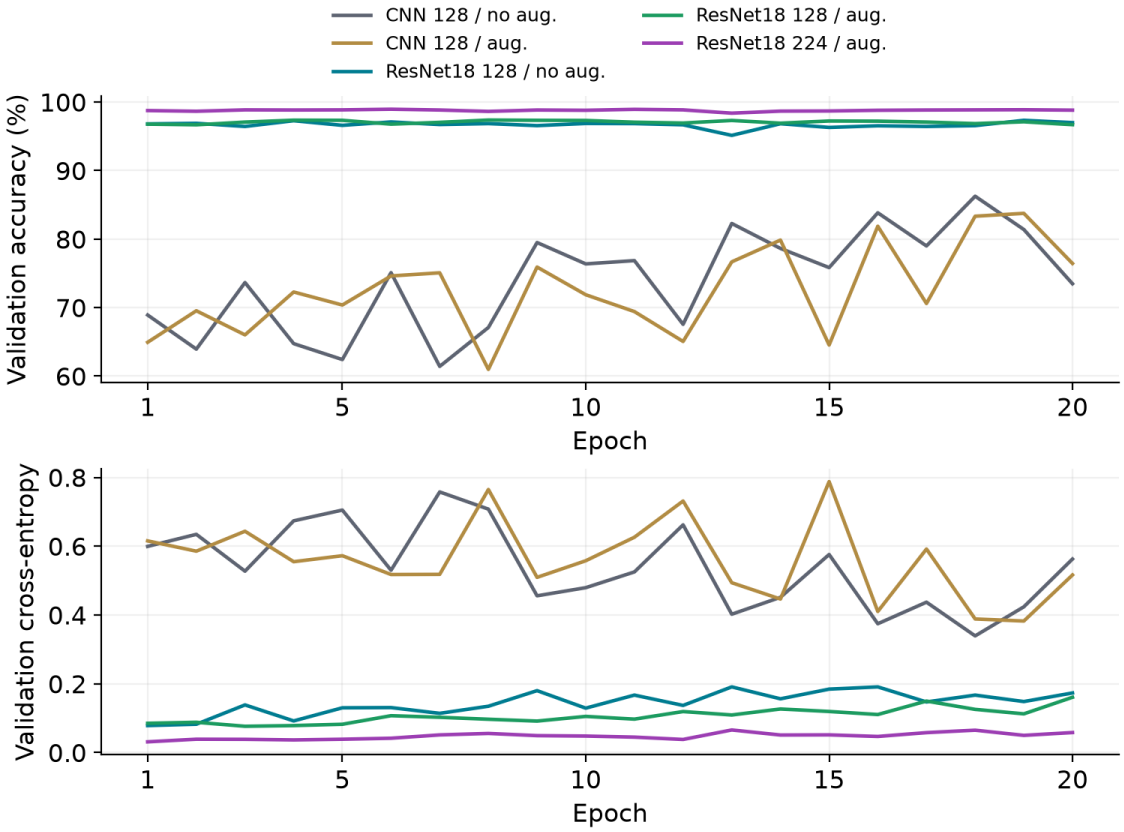


Figure 4.2 Validation accuracy and loss across all 20 epochs. R2 is a separate resolution experiment.

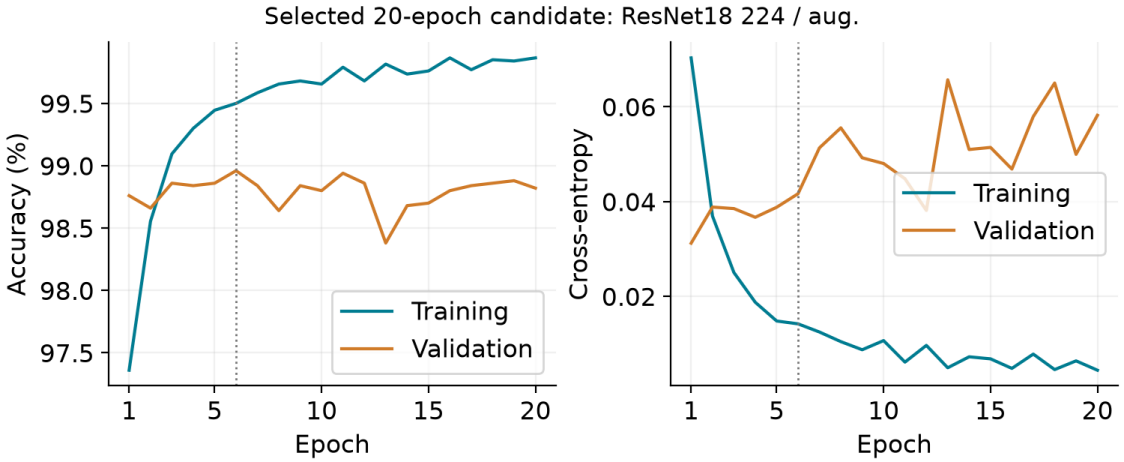


Figure 4.3 Training and validation trajectories for the selected candidate (ResNet18 224 / aug.); the dotted line marks selected epoch 6.

The selected checkpoint is epoch 6 (validation accuracy 98.96%, loss 0.0417). At epoch 20, training accuracy is 99.87%, validation accuracy is 98.82% and validation loss is 0.0582. All 20 epochs were completed; selection follows the stated rule rather than always taking the last checkpoint. Training metrics accumulate while weights change and training augmentation is applied; validation uses the post-epoch model and deterministic inputs.

5. Validation Accuracy and Test Predictions (d)

5.1 B extension: validation results and checkpoint selection

Run	Aug.	Input	Best epoch	Val. acc.	Val. loss
C0	No	128	18	86.26%	0.3392
C1	Yes	128	19	83.76%	0.3824
R0	No	128	19	97.34%	0.1482
R1	Yes	128	8	97.38%	0.0968
R2	Yes	224	6	98.96%	0.0417

Table 5.1 New 20-epoch results (n = 5,000 validation images); loss is measured at the selected checkpoint. Run order is C0, C1, R0, R1 and R2 as defined in Table 4.2.

The selected B candidate is ResNet18 224 / aug., selected from the three new ResNet18 runs using validation accuracy (then loss) alone. Its independently reloaded checkpoint correctly classifies 4,948 of 5,000 validation images: accuracy 98.96%, macro-F1 0.9896 and cross-entropy 0.0417. Reloaded accuracy exactly matches the recorded checkpoint accuracy. All five completed runs have 20 history rows; their saved best checkpoints were evaluated independently.

Class	Precision	Recall	F1	Support
cat	0.9935	0.9856	0.9896	2,500
dog	0.9857	0.9936	0.9896	2,500

Table 5.2 Per-class metrics for the selected B checkpoint; cat = 0, dog = 1.

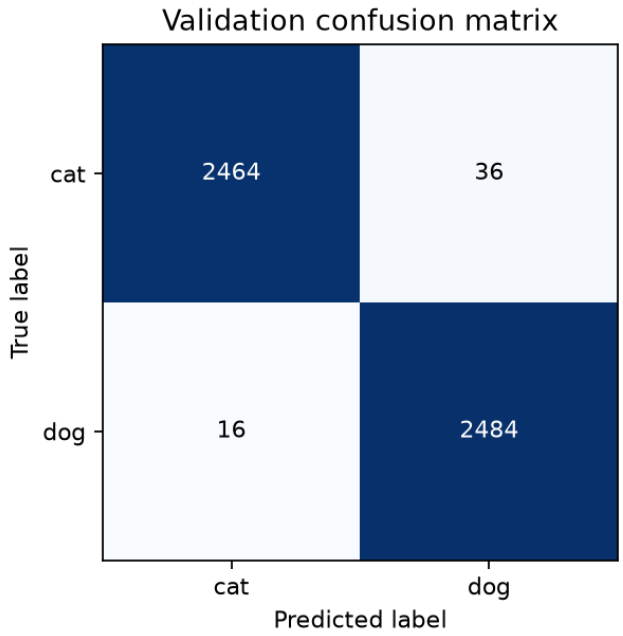


Figure 5.1 Selected B checkpoint confusion matrix. Rows are ground-truth labels; columns are predictions.

After excluding val/cat/cat.12408.jpg from validation scoring only, accuracy is 98.9598% on 4,999 images. This sensitivity score quantifies the effect of the one known exact duplicate; the official 5,000-image result remains the main protocol. The absence of a fully deduplicated retraining or a separate holdout limits any claim about unseen-data performance.

5.2 B extension: test prediction and submission format

The selected checkpoint was applied once to the 500 unlabelled course test images after validation-based selection. The new submission.csv contains exactly two columns, id and label, in the supplied sample order (IDs 1–500), with integer labels 0 = cat and 1 = dog. It contains 241 cat predictions and 259 dog predictions. A separate test_predictions.csv stores dog probabilities; probabilities do not replace the discrete submission labels. The checkpoint digest, selected epoch, manifest digest and row count are recorded in provenance.json. No test accuracy is available because test labels are not provided.

Selected run: resnet224_aug; checkpoint: experiments/resnet224_aug/best_model.pt; checkpoint epoch: 6. New test output: selected_submission/submission.csv in the B20 evidence package. These new outputs are separate from the earlier 10-epoch submission, which is retained unchanged.

6. Correct and Incorrect Sample Analysis (e)

6.1 Overall validation behaviour

The epoch-18 checkpoint classified 4,167 of the 5,000 validation images correctly, giving an accuracy of 83.34%. It correctly identified 2,120 cats and 2,047 dogs. Recall was 84.80% for cats and 81.88% for dogs, so the remaining errors were distributed across both classes, with dogs missed slightly more often.

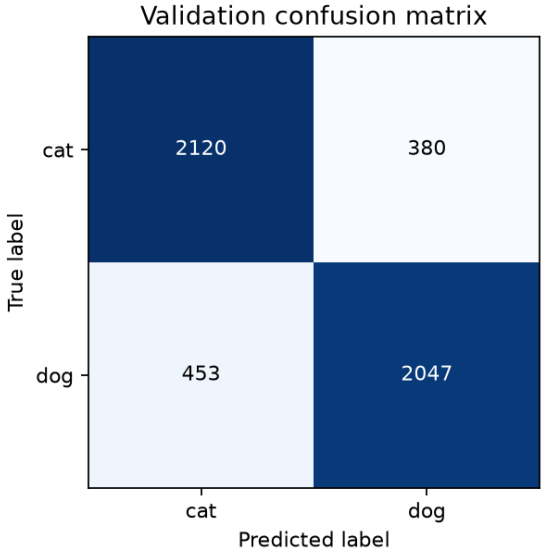


Figure 6.1 Validation confusion matrix at the selected checkpoint

6.2 Selected correct and incorrect cases



Figure 6.2 One correct and one incorrect validation example

In dog.10708.jpg, the dog occupies most of the frame and has a clear outline against a fairly uniform grass background. Its coat pattern and body shape remain visible after resizing, and the model predicts dog with 0.821 confidence. This is the kind of image the small CNN handles well because the main subject is prominent and the visual cues survive the reduced resolution.

cat.10657.jpg is much harder. The image is dark and blurred, the cat is relatively small, and little facial or fur detail remains after resizing to 128 × 128. The model predicts dog with 0.551 confidence, only just above the decision threshold. Low contrast and lost detail may have contributed to the error, although one image is not enough to prove the cause. Testing a larger input size, brightness augmentation or a pretrained network would help determine which factor matters most.

6.3 B extension: labelled validation cases

The following examples come from the independently reloaded, newly selected 20-epoch checkpoint. Ground truth is available for validation, so correct and incorrect classifications can be verified. The cases illustrate behavior rather than estimate its frequency; overall frequencies are given by the confusion matrix.



Figure 6.3 Two correct and two incorrect validation cases from the selected B checkpoint. The displayed confidence is the predicted-class softmax probability.

cat.10868.jpg is correctly classified: ground truth cat, prediction cat, predicted-class confidence 1.0000. The black cat occupies much of the image, with clear pointed ears, eyes and whiskers. These visible facial cues and the fairly simple background make this an interpretable success case.

dog.1.jpg is correctly classified: ground truth dog, prediction dog, predicted-class confidence 1.0000. The brown dog is shown almost in full, facing the camera. Its muzzle, ears and body remain visible despite the cage bars at the side. This case shows that some background clutter can be tolerated when the animal itself is clear.

cat.11565.jpg is misclassified: ground truth cat, prediction dog, predicted-class confidence 0.9999. This is a stylized cat graphic with the text “PHOTO UNAVAILABLE”, rather than a natural animal photograph. It is a dataset artifact and a different visual domain from most training images. The high-confidence mistake illustrates that a softmax score is not a guarantee of correctness; calibration and out-of-distribution rejection were not evaluated.

dog.3970.jpg is misclassified: ground truth dog, prediction cat, predicted-class confidence 0.9694. The small light-coloured dog has pointed ears and a relatively compact body, with a large ball beside its head. Cat-like shape cues and the nearby object may make the image ambiguous at the model input resolution. This is a plausible interpretation, not a causal explanation established by an attribution experiment.

6.4 B extension: qualitative inspection of unlabelled test images

The assignment asks for case analysis on the test set, but the supplied test images have no labels. The two test examples below therefore discuss visual plausibility and potential failure modes without labelling them as verified correct or incorrect. The labelled validation cases above provide the measurable error analysis.



Figure 6.4 Two unlabelled test predictions. Visual inspection is qualitative and is not ground-truth evaluation.

Test ID 92 is predicted as cat, with P(dog) = 1.97999e-33. The close-up contains clear cat-like ears, whiskers and facial structure with little occlusion. The cat prediction is visually plausible. No official ground-truth label is available, so this observation is not counted as verified correct or incorrect.

Test ID 245 is predicted as dog, with P(dog) = 0.649596. A brown animal is partly hidden by thick white cage bars, and the face is not clearly visible. This obstructed view provides fewer reliable category cues than the close-up in ID 92. The image is useful for illustrating occlusion as a potential failure mode; its probability must not be treated as a calibrated likelihood of correctness. No official ground-truth label is available, so this observation is not counted as verified correct or incorrect.

7. Model and Data Processing Comparison (f)

7.1 B extension: model and augmentation effects at 128 × 128

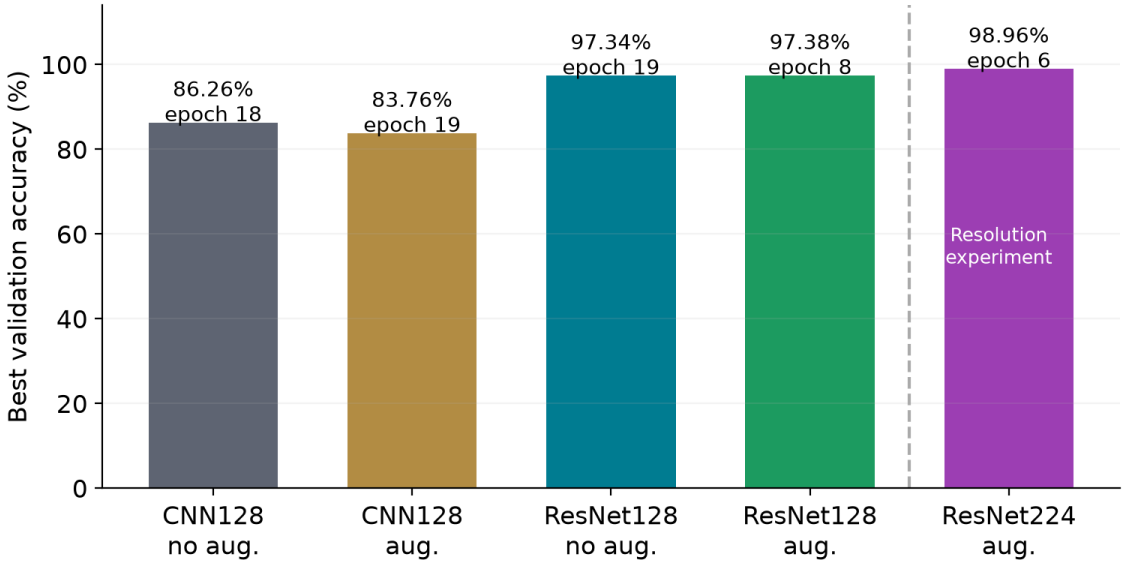


Figure 7.1 Best validation accuracy under a common 20-epoch budget. The first four bars form the controlled 128 × 128 comparison; the fifth changes resolution.

At 128 × 128 without augmentation, changing from the scratch CNN to the pretrained, partially fine-tuned ResNet18 changes accuracy by +11.08 percentage points. With augmentation, the corresponding change is +13.62 points. This is evidence for the tested model-and-transfer-learning configuration under this protocol, rather than proof that architecture alone causes the improvement: pretraining, model capacity and optimization policy also change.

The augmentation effect at 128 × 128 is -2.50 percentage points for the CNN and +0.04 points for ResNet18. Consequently, augmentation should be judged from the observed matched results rather than assumed to help every model. Rotation and flipping may improve invariance but also alter the effective training distribution; one fixed-seed pair does not identify a universal augmentation effect.

The CNN validation curves fluctuate substantially. At epoch 20, C0 and C1 reach 73.48% and 76.44%, respectively, below their selected best checkpoints. Reporting the selection rule and full trajectories is therefore essential; the last checkpoint must not be silently substituted for the best one. Completing 20 epochs does not by itself establish convergence. These observations do not establish whether longer training or a different augmentation strength would help.

7.2 B extension: input resolution and relation to the A baseline

Within augmented ResNet18, raising input resolution from 128 × 128 to 224 × 224 changes validation accuracy by +1.58 percentage points. The larger input preserves finer details but increases computation. The summed recorded training/validation epoch times are 9.4 minutes at 128 and 14.6 minutes at 224; these are same-machine measurements excluding setup, checkpoint writing and plotting, not deployment latency benchmarks.

The original A baseline reported 83.34% at epoch 18 of a 20-epoch run, as preserved in Sections 4 and 6. The new 20-epoch augmented CNN control reaches 83.76%. It is a separate run, not a replacement for A's reported result. A used Python 3.11.9 and an RTX 5060 Laptop GPU; the new controlled runs used Python 3.12.3 and an RTX 2060, with four data-loader workers. The first four new controls share the same environment, split, seed, resolution, batch size and training budget; this is the main comparison used to discuss model and augmentation choices.

7.3 B extension: reproducibility and limitations

The new 20-epoch matrix uses one fixed seed per setting. It supports a controlled, reproducible comparison but does not estimate variation across seeds or establish statistical significance. The earlier augmented 224 × 224 ResNet18 exploration used seeds 42, 43 and 44 for 10 epochs each, producing 98.96%, 98.88% and 98.86% (mean 98.90%, sample standard deviation 0.053 percentage points). That variability estimate belongs only to the earlier 10-epoch protocol and must not be attributed to the new 20-epoch runs. Validation is reused for tuning and checkpoint selection; a dedicated nested selection split was not used.

Further limits are external ImageNet supervision, the known exact train/validation duplicate, possible perceptual duplicates, square resizing, and unavailable test labels. A scratch ResNet18 control was not trained, so the benefit of pretraining cannot be separated from capacity. Accuracy and macro-F1 are balanced-class measures for this split; confidence values have not been calibrated. The new results do not establish open-set robustness or performance under a shifted deployment distribution.

7.4 B contribution and handoff

ZHU YANGKE — B module: implemented the isolated pretrained ResNet18 classifier and freeze policy; reused the shared data/training/evaluation APIs; conducted model, augmentation and resolution studies; completed the five matched 20-epoch controls; verified best-checkpoint reloads; generated validation metrics, figures and the new test submission; and prepared the B methodology, results, case analysis and reproducibility material in this report. Existing A and C material is retained. The group literature survey and C-module sections remain for team integration. This updated report and the new evidence package have not been merged into main.

7.5 References for the B extension

[B1] K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” Proceedings of CVPR, 2016, pp. 770–778. https://arxiv.org/abs/1512.03385

[B2] PyTorch / torchvision, “resnet18 — ResNet18_Weights.IMAGENET1K_V1,” torchvision 0.29 documentation, accessed 24 October 2026. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

B experiment update: 24 October 2026. Original cover and A/C text are retained from the supplied report; the cover date therefore remains the original draft date.

8. CIFAR-10 Multi-category Classification (g)

9. CIFAR-10 Class Imbalance (h)